# Optimización mediante Algoritmos Genéticos de la asignación y el balance de carga de alertas de seguridad en Centros de Operaciones de Seguridad (SOC)

**Abstract**

*El presente trabajo aborda el problema de asignación y scheduling de alertas en un Centro de Operaciones de Seguridad (SOC) mediante un Algoritmo Genético con selección por ranking, elitismo y operadores adaptados a la representación utilizada. El volumen de alertas puede exceder la capacidad humana, generando sobrecarga y riesgo de incumplimiento de los Acuerdos de Nivel de Servicio (SLA) para incidentes críticos. Se desarrolló una función objetivo ponderada que combina tiempo de resolución, espera, cumplimiento de SLA, backlog y balance de carga entre analistas. La evaluación utiliza alertas derivadas del dataset público CICIDS2017 y complementadas con atributos operativos sintéticos. El algoritmo se compara con Round-Robin, Menor carga y Urgencia balanceada; esta última utiliza la misma información operativa sin realizar búsqueda evolutiva. Los resultados muestran compromisos entre métricas: el AG obtiene la menor espera crítica en la corrida de referencia, mientras que Urgencia balanceada logra el mejor desbalance, por lo que no se observa una superioridad universal.*

**Palabras Clave**

Algoritmos Genéticos, Centro de Operaciones de Seguridad (SOC), Scheduling, Asignación de Tareas, Balance de Carga, CICIDS2017.

## Introducción

Los Centros de Operaciones de Seguridad (SOC) reciben un volumen de alertas que crece con mayor velocidad que la disponibilidad de analistas capacitados para atenderlas. La experiencia y la literatura especializada coinciden en que la sobrecarga (*alert overload*) es uno de los problemas estructurales más persistentes de la operación de un SOC, ya que obliga a decidir con información incompleta y bajo presión de tiempo qué atender primero.

Esa sobrecarga tiene un correlato humano bien documentado en la industria: la fatiga de alertas (*alert fatigue*). Un relevamiento sistemático reciente informa que la mayoría de las organizaciones recibe más de 10.000 alertas diarias y que más del 50 % son falsos positivos [3]. En este contexto, los analistas pueden llegar a ignorar alertas, desactivarlas o delegarlas a colegas, en lugar de investigarlas [3]. La fatiga degrada la calidad de las decisiones de triaje, incrementa el tiempo de respuesta y favorece la rotación del personal capacitado. El resultado es un ciclo que se retroalimenta: más alertas, analistas menos efectivos y tiempos de respuesta más largos, con el agravante de que la demora en atender una alerta de alta severidad incrementa el riesgo de que una intrusión progrese sin ser contenida.

A esta escasez estructural se suma un problema de coordinación: aun cuando el equipo de analistas es suficiente en promedio, la forma en que las alertas se reparten entre ellos rara vez es óptima. Las prácticas habituales de asignación —turnos fijos, reglas estáticas por tipo de alerta o asignación manual según disponibilidad aparente— no consideran de forma simultánea la prioridad de cada alerta, el plazo de resolución comprometido (SLA), el tiempo estimado que insumirá resolverla y la carga ya acumulada por cada analista. El resultado observado en la práctica es un desbalance de carga: algunos analistas acumulan *backlog* mientras otros permanecen subutilizados.

El problema fundamental es que la asignación manual no logra equilibrar de forma consistente el tiempo total de resolución, los tiempos de espera de las alertas críticas y el desbalance de carga. Por ello, la presente investigación propone diseñar, implementar y evaluar un modelo de Algoritmo Genético (AG) que distribuya las alertas —caracterizadas por su prioridad, severidad, tiempo estimado de resolución y SLA— en-

tre un número limitado de analistas mediante una función objetivo ponderada.

Este planteo se relaciona con la priorización de alertas estudiada por Jalalvand et al. [3], quienes organizan los criterios relevantes en dimensiones asociadas con la alerta, el analista, el activo, la organización y el entorno externo. En particular, su relevamiento destaca la necesidad de combinar criterios técnicos con restricciones de recursos humanos y de tiempo, en lugar de ordenar las alertas únicamente por severidad.

**Elementos del Trabajo y metodología**

La metodología propuesta se enmarca en la resolución de un problema de asignación y scheduling en máquinas paralelas mediante un Algoritmo Genético. La instancia puede interpretarse como un caso particular relacionado con el *Job Shop Scheduling* estocástico, ya que cada alerta se modela como una tarea asignada a un analista y no posee una ruta de múltiples operaciones.

La asignación de tareas con tiempos de procesamiento y recursos limitados pertenece al scheduling de máquinas paralelas y se relaciona con variantes estocásticas del *Job Shop Scheduling* (SJSP) [4]. En este trabajo cada alerta es una tarea de una sola operación y los analistas actúan como máquinas paralelas; por eso el JSP se utiliza como marco de referencia y no como clasificación estricta. Los tiempos nominales se complementan con duraciones lognormales de media igual al tiempo estimado y coeficiente de variación configurable. El AG optimiza el valor esperado sobre escenarios comunes y la solución se valida con escenarios nuevos fuera de muestra.

Se diseñó una representación cromosómica específica con tres bloques: asignación de alertas a analistas, ajuste de prioridad de despacho y una política global de criticidad. El primer bloque contiene un gen por alerta y su alelo indica el analista responsable; el segundo bloque contiene un ajuste acotado que modifica el orden base; el tercero contiene dos genes globales $(w_2, w_3)$, acotados, que determinan cuánto utiliza la política evolucionada la criticidad de activos importantes y críticos. Las variantes reducidas desactivan el bloque de ajuste o el de criticidad, respectivamente.

Esta codificación directa resulta adecuada para representar la decisión principal del problema —qué analista recibe cada alerta— y evita introducir una capa de codificación indirecta. El segundo bloque permite evolucionar una política de despacho específica por alerta y los dos genes globales permiten interpretar directamente la sensibilidad aprendida para cada nivel de criticidad, sin duplicar genes por alerta ni alterar los datos observados de la instancia. Boukedroun et al. [4] reportan que las codificaciones directas pueden representar el cronograma en la estructura de datos y reducir la necesidad de mecanismos complejos de reparación, criterio que se adopta aquí mediante alelos acotados al conjunto de analistas disponibles y ajustes limitados.

La función de evaluación o *fitness* fue formulada para buscar el menor impacto temporal y el mayor equilibrio, combinando el tiempo total estimado de resolución con penalizaciones estrictas por:

- Tiempos de espera prolongados.
- Incumplimiento de SLA (especialmente en alertas críticas).
- Desbalance de carga entre los analistas operativos.

Formalmente, el algoritmo minimiza una única función objetivo escalarizada $J$, que pondera varios criterios operativos, y transforma su resultado en una aptitud positiva a maximizar:

$$\begin{aligned} J &= 0.10\,\widehat{T} + 0.10\,\widehat{W} + 0.20\,\widehat{W}_c + 0.25\,\widehat{R}_c \\ &\quad + 0.10\,\widehat{B}_p + 0.05\,\widehat{B}_m + 0.15\,D + 0.05\,O \\ &\quad + 0.25\,\widehat{S}_a, \\ f &= \frac{1}{1+J}. \end{aligned} \tag{1}$$

Aquí, las variables con sombrero representan magnitudes normalizadas: $\widehat{T}$ es el makespan relativo al horizonte, $\widehat{W}$ la espera promedio, $\widehat{W}_c$ la espera crítica, $\widehat{R}_c$ el retraso crítico respecto del SLA efectivo, $\widehat{B}_p$ el backlog ponderado por prioridad, $\widehat{B}_m$ los minutos de backlog, $D$ el desbalance relativo de carga, $O$ la sobrecarga relativa y $\widehat{S}_a$ la violación de SLA ponderada por prioridad y criticidad del activo. La configuración oficial utiliza $w_T = 0.10$ y el término de activo tiene peso 0.25.

El algoritmo evoluciona la población generación tras generación aplicando selección por ranking, cruzamiento y mutación por bloques para recombinar asignaciones y ajustes de prioridad, además de elitismo. La población inicial incluye una solución Round-Robin con ajuste nulo como referencia y la política de atención dinámica prioriza las alertas disponibles según prioridad, severidad, ajuste evolucionado y política global de criticidad. La selección por torneo y por ruleta se conservan como alternativas reproducibles. Estos operadores forman parte del marco clásico de planes reproductivos y operadores genéticos desarrollado por Holland, que también analiza la robustez de los planes evolutivos y el papel de las codificaciones en sistemas adaptativos [2].

La configuración oficial utiliza una población de $N = 50$ individuos y evoluciona durante $G = 200$ generaciones, es decir, hasta 10.000 evaluaciones de individuos por corrida. La probabilidad de cruzamiento de un punto es $P_c = 0.75$ y la probabilidad de mutación por gen es $P_m = 0.05$. Se preservan los tres mejores individuos por generación y se utiliza selección por ranking. El escenario principal contiene 100 alertas, diez analistas, un horizonte de 480 minutos y criticidad de activos derivada de un catálogo CMDB simulado. Se agregan un escenario de estrés con 120 alertas y un escenario de saturación con 500 alertas. Cada escenario se evalúa sobre 20 semillas (42–61), comparando el AG y Round-Robin con las mismas alertas y los mismos escenarios de duración para cada semilla. La ablación propuesta separa cuatro variantes: A (solo asignación), B (asignación y $d_i$), C (asignación y $(w_2, w_3)$) y D (todos los bloques). El grid search de 18 combinaciones se utilizó únicamente como exploración de hiperparámetros, con 15 generaciones por combinación; su mejor configuración ($N = 20$, $P_m = 0.05$, $P_c = 0.60$) no reemplaza la configuración oficial $N = 50$, $G = 200$.

Para validar empíricamente el modelo, se utilizó una base de datos representativa derivada del dataset público **CICIDS2017** [1]. A partir de una muestra aleatoria de eventos, se inyectaron sintéticamente atributos operativos (prioridad, severidad, SLA y tiempo estimado de resolución). La criticidad del activo se generó mediante un catálogo CMDB simulado, con 60 % de activos comunes, 30 % importantes y 10 % críticos. Esta decisión permite estudiar el término de criticidad aunque el dataset original no contiene inventario de activos; por lo tanto, esos valores no deben interpretarse como observaciones reales de la infraestructura.

La selección de estos atributos está alineada con la taxonomía de criterios de priorización de Jalalvand et al. [3]: prioridad y severidad representan propiedades de la alerta; el tiempo de resolución aproxima el tiempo disponible y la carga del analista; y el SLA expresa la dimensión organizacional de cumplimiento y oportunidad (*timeliness*). En lo que sigue, "alerta crítica" refiere a una alerta marcada por su prioridad o severidad, mientras que "activo crítico" refiere exclusivamente a la categoría de criticidad obtenida de la CMDB simulada. La criticidad observada del activo se integra en el cromosoma mediante el tercer bloque global $(w_2, w_3)$: no se modifica artificialmente la criticidad de la instancia, sino que se aprende cuánto debe influir cada nivel en el orden de despacho. El orden efectivo resta $n \cdot 0.5 \cdot w_2$ a los activos importantes y $n \cdot 0.5 \cdot w_3$ a los críticos. Además, la criticidad interviene en la función de penalización mediante el SLA efectivo y el peso de la violación. Esta correspondencia ofrece una justificación metodológica para la

función de aptitud, aunque los atributos hayan sido generados sintéticamente.

Cada escenario genera una tabla de alertas derivadas con el tamaño indicado (100, 120 o 500 registros). La distribución de prioridades, la severidad y el tiempo estimado se calculan nuevamente para cada muestra y semilla; estos valores describen las instancias experimentales y no deben interpretarse como propiedades originales del dataset CICIDS2017.

Como baselines operativos se utilizaron tres reglas deterministas. Round-Robin asigna las alertas secuencialmente entre los analistas, sin considerar carga ni prioridad. Menor carga asigna cada alerta al analista con menor carga acumulada. Urgencia balanceada ordena las alertas por prioridad, severidad y SLA, y asigna cada una al analista con menor carga; por lo tanto, utiliza la misma información operativa que el AG sin realizar búsqueda evolutiva. Esta última comparación permite distinguir, aunque sea en una instancia de referencia, el efecto de una política de despacho del efecto de la búsqueda evolutiva. Estas reglas no forman parte del entrenamiento del AG.

Además del esquema por lote, el prototipo implementa un flujo continuo mediante un horizonte rodante. En cada intervalo de replanificación se reoptimiza únicamente el conjunto de alertas ya llegadas y aún no iniciadas; las tareas que comienzan dentro del intervalo quedan comprometidas y se ejecutan con duraciones reales desconocidas por el planificador. Esta modalidad puede compararse con el AG por lote y Round-Robin bajo la misma realización de duraciones. La ejecución continua se utiliza como evaluación operativa complementaria y no reemplaza la comparación multi-semilla del escenario por lote.

## Resultados

Las ejecuciones del algoritmo se evaluaron mediante métricas operativas y se compararon con una asignación Round-Robin. La comparación multi-semilla utiliza medias, desvíos estándar, intervalos de confianza bootstrap y porcentaje de semillas en las que el AG mejora. La validación fuera de muestra usa escenarios de duración nuevos y compartidos por ambas soluciones, de modo que la comparación no favorece al método que optimiza.

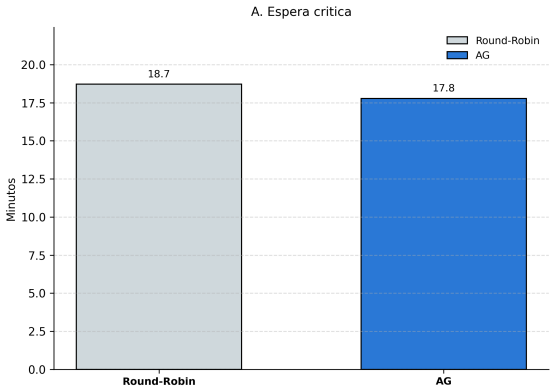



**Figura 1:** Espera crítica promedio para la corrida reproducida con la semilla 42.

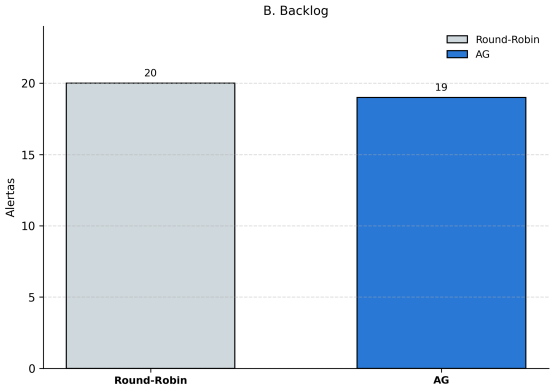



**Figura 2:** Backlog total para la corrida reproducida con la semilla 42.

Las figuras ilustran una corrida puntual y no se utilizan como evidencia suficiente de superioridad. Las conclusiones cuantitativas se extraen de `outputs/experimento/experimento_resumen.csv`, que separa los escenarios principal, de estrés y de saturación. En el escenario principal, el AG obtuvo un fitness medio de 0.8702 frente a 0.8599 de Round-Robin, redujo la espera crítica de 13.25 a 10.37 minutos y el desbalance de 0.1485 a 0.0948. Sin embargo, el cumplimiento global de SLA fue 89.78 % frente a 89.94 % y el backlog fue 15.90 frente a 15.36 alertas.

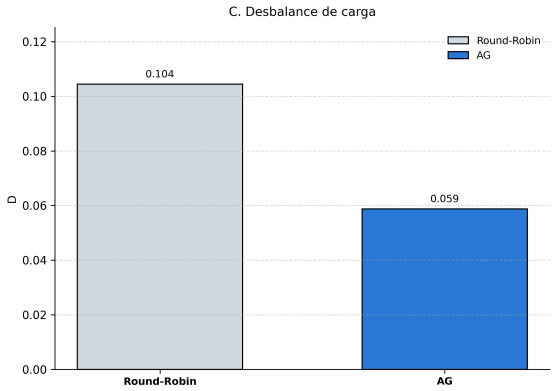



**Figura 3:** Desbalance relativo de carga para la corrida reproducida con la semilla 42.

En el escenario de estrés, con 120 alertas, el AG mantuvo ventajas en fitness (0.8504 frente a 0.8414), espera crítica (17.02 frente a 19.82 minutos) y desbalance (0.0884 frente a 0.1260), pero presentó menor cumplimiento global de SLA (78.07 % frente a 79.34 %) y mayor backlog (30.54 frente a 29.54 alertas). El cumplimiento de SLA de alertas críticas fue superior para el AG en ambos escenarios no saturados: 84.47 % frente a 79.82 % en el escenario principal y 74.43 % frente a 69.57 % en el de estrés.

El escenario de 500 alertas está deliberadamente sobredimensionado respecto del horizonte y se interpreta como saturación, no como escenario operativo principal. Allí el AG obtuvo un fitness menor (0.4333 frente a 0.4442) y una espera crítica mayor (269.82 frente a 182.96 minutos), aunque redujo el backlog (416.81 frente a 421.57 alertas). El desbalance fue ligeramente menor para el AG (0.0521 frente a 0.0552), pero la diferencia no fue concluyente. El cumplimiento global de SLA fue 9.72 % frente a 9.04 %. Por lo tanto, el escenario saturado confirma que el backlog y los tiempos quedan dominados por la insuficiencia estructural de capacidad.

A partir del análisis generacional se observaron además las siguientes medidas:

- **Fitness y estabilidad:** el AG superó a Round-Robin en fitness en las 20 semillas de los escenarios principal y de estrés, mientras que fue inferior en saturación; las medias fueron 0.8702, 0.8504 y 0.4333 para 100, 120 y 500 alertas, respectivamente. Frente a los baselines deterministas, Menor carga obtuvo mayor fitness medio en los tres escenarios (0.8788, 0.8586 y 0.4506), por lo que la ventaja del AG no puede atribuirse únicamente a la búsqueda evolutiva.

- **Cumplimiento de SLA:** el AG mejoró el cumplimiento de alertas críticas en los escenarios principal y de estrés, pero no en saturación; el cumplimiento global fue menor en los dos primeros y mayor en saturación. En activos críticos no hubo diferencia clara en los escenarios no saturados, mientras que el AG fue superior en saturación.

- **Espera y balance:** el AG redujo en promedio la espera crítica y el desbalance frente a Round-Robin en los escenarios principal y de estrés. En saturación aumentó la espera crítica y la diferencia de desbalance no fue concluyente. En la corrida de referencia, Urgencia balanceada obtuvo menor desbalance que el AG.

- **Capacidad:** el backlog se analiza separando los escenarios no colapsados de la saturación de 500 alertas.

El archivo de distribución final muestra las cargas asignadas de la corrida oficial. En el grid search de 18 configuraciones, la mejor aptitud fue obtenida con población 20, mutación 0.05 y cruzamiento 0.60. La configuración oficial utiliza una población mayor y más generaciones para realizar una búsqueda más estable y profunda que la exploración preliminar. En la corrida de referencia, la comparación de baselines mostró que la regla de urgencia balanceada puede distribuir mejor la carga, mientras que el AG obtiene una espera crítica menor:

La tabla no muestra un ganador universal: el AG obtiene la menor espera crítica en esta corrida de referencia, Menor carga el menor backlog y Urgencia balanceada el menor

**Cuadro 1:** Comparación de baselines en la corrida de referencia.

| Método | Espera cr. | Backlog | Desbalance de carga | Espera prom. |
|---|---|---|---|---|
| Round-Robin | 18.71 | 20 | 0.1044 | 39.06 |
| Menor carga | 19.14 | 15 | 0.0321 | 31.36 |
| Urgencia balanceada | 19.68 | 27 | 0.0081 | 67.29 |
| AG | 17.79 | 19 | 0.0587 | 38.79 |

**Cuadro 2:** Espera crítica media en la comparación multi-semilla de baselines.

| Método | Principal | Estrés | Saturación |
|---|---|---|---|
| Round-Robin | 13.25 | 19.82 | 182.96 |
| Menor carga | 9.24 | 17.06 | 167.44 |
| Urgencia balanceada | 19.80 | 23.67 | 157.65 |
| AG | 10.37 | 17.02 | 269.82 |

desbalance, pero con mayor espera promedio. Esta tabla corresponde a una corrida de referencia y debe leerse junto con la comparación multi-semilla.

Para evitar que una sola semilla determine la interpretación, se repitió la comparación de los tres baselines en las mismas 20 semillas y escenarios de validación:

Menor carga fue mejor que el AG en espera crítica en el escenario principal (9.24 frente a 10.37 minutos), aunque la diferencia no fue concluyente tras Holm ($p_{\text{Holm}} = 0.172$); en estrés los valores fueron prácticamente iguales (17.06 frente a 17.02; $p_{\text{Holm}} = 1.000$). En saturación, Menor carga redujo la espera crítica de forma concluyente (167.44 frente a 269.82; $p_{\text{Holm}} < 0.001$) y también obtuvo menor backlog y desbalance. Urgencia balanceada tuvo mayor espera crítica que el AG en los escenarios principal y de estrés, pero menor espera en saturación a costa de un backlog mayor. Los resultados completos, incluidos los intervalos bootstrap y las pruebas de Wilcoxon con Holm, se encuentran en `outputs/baselines_multisemilla/`.

Como control adicional se ejecutó una búsqueda aleatoria con el mismo presupuesto computacional del AG: 10.000 evaluaciones por combinación de escenario y semilla. En cada caso se utilizaron la misma instancia, los mismos escenarios de duración y la misma validación fuera de muestra. El fitness medio del AG fue 0.8702, 0.8504 y 0.4333 en los escenarios principal, de estrés y de saturación, frente a 0.8478, 0.8265 y 0.4105 de la búsqueda aleatoria, respectivamente. La espera crítica media también fue menor para el AG en los escenarios principal y de estrés: 10.37 frente a 18.68 minutos y 17.02 frente a 27.59 minutos; en saturación también fue menor para el AG (269.82 frente a 364.70 minutos). Este control aporta evidencia de que el AG encuentra mejores soluciones según su función objetivo que una búsqueda aleatoria con igual presupuesto, aunque no demuestra superioridad en todas las métricas ni frente a toda heurística alternativa.

Los pesos de la función objetivo se mantienen fijos antes de observar los resultados. No se modifican para forzar una ventaja del AG sobre Round-Robin; si existe un compromiso entre balance, espera crítica y SLA, se informa explícitamente.

La comparación de ablación se realizó de forma pareada por escenario y semilla, con intervalos bootstrap, prueba de Wilcoxon y corrección de Holm; los resultados completos se guardan en `outputs/comparacion_ablacion.csv`.
En el escenario principal no se detectaron diferencias estadísticamente concluyentes entre las variantes en las métricas principales. En el escenario de estrés, C y D aumentaron el backlog frente a A en aproximadamente 1.49 y 1.39 alertas, respectivamente ($p_{\text{Holm}} = 0.040$ y 0.035), sin una mejora concluyente de fitness, espera crítica o desbalance. Bajo saturación, B, C y D redujeron la espera crítica frente a A en aproximadamente 15.5, 162.2 y 119.4 minutos, respectivamente (en todos los casos, $p_{\text{Holm}} < 0.001$), pero redujeron el fitness en 0.0032, 0.0237 y 0.0184 y aumentaron el backlog en 0.94, 7.13 y 6.68 alertas, respectivamente. D además redujo el desbalance frente a A en aproximadamente 0.006 ($p_{\text{Holm}} = 0.010$). Estos resultados muestran un compromiso: la política de criticidad puede reducir esperas críticas bajo

sobrecarga, pero no mejora globalmente la función objetivo.

Los pesos globales $(w_2, w_3)$ son más fáciles de interpretar en saturación, donde realmente cambian la espera de las alertas críticas. En los escenarios principal y de estrés varios pesos llegaron cerca del techo de 0.98, sin producir un efecto estadísticamente detectable. Por eso, los valores altos no significan por sí solos que el algoritmo haya descubierto una regla nueva: son coherentes con que la función objetivo ya da más importancia a los activos críticos. Así, la incorporación de la criticidad dentro del cromosoma se justifica principalmente para situaciones de sobrecarga; en condiciones normales, su efecto es limitado.

Los intervalos bootstrap del resumen permiten distinguir efectos consistentes de resultados exploratorios. Aunque el AG supera a Round-Robin en varias medias de fitness, espera crítica y desbalance, esas diferencias no deben interpretarse automáticamente como concluyentes. El efecto sobre el cumplimiento global de SLA y el backlog debe informarse junto con cualquier mejora en la espera crítica.

Este análisis tiene dos limitaciones. Se realizaron 105 pruebas estadísticas, 35 por escenario, y la corrección de Holm se aplicó dentro de cada escenario; por lo tanto, los resultados deben interpretarse como evidencia por escenario y no como una prueba única para todas las comparaciones. Además, 20 semillas ofrecen poca potencia para detectar efectos pequeños, por lo que una diferencia no significativa no demuestra que las variantes sean exactamente iguales.

## Discusión

Los resultados obtenidos muestran que la función de aptitud multifactorial permite mejorar algunos objetivos operativos frente a Round-Robin, pero no todos simultáneamente. En los escenarios principal y de estrés, el AG reduce la espera de alertas críticas y el desbalance, a costa de un mayor backlog y menor cumplimiento global de SLA. En saturación, en cambio, el AG presenta mayor espera crítica y menor fitness, aunque reduce el backlog.

A diferencia de los enfoques estáticos manuales, el Algoritmo Genético permite expresar en una única función objetivo el tiempo de resolución, la espera crítica, el backlog y el balance de carga. Sin embargo, la comparación de esta corrida no muestra una superioridad automática frente a Round-Robin en todas las métricas. Esto es consistente con la observación de Jalalvand et al. [3] de que los métodos de optimización permiten incorporar explícitamente restricciones como la disponibilidad de analistas y el tiempo de investigación, aunque permanecen menos explorados que los enfoques basados en aprendizaje automático. En consecuencia, el modelo propuesto se interpreta como una forma de Alert Prioritisation (AP) automatizada para el triaje y la asignación inicial, no como un reemplazo de la validación experta ante incidentes nuevos o ambiguos.

El backlog total no debe leerse de la misma manera en todos los escenarios. En la saturación de 500 alertas, la demanda supera ampliamente la capacidad del turno y el backlog queda dominado por ese límite estructural. En los escenarios de 100 y 120 alertas, en cambio, el aumento de backlog del AG sí representa un costo de la política de priorización y debe informarse junto con la mejora en alertas críticas.

La evaluación fuera de muestra evita que la mejora observada sea atribuida únicamente a los escenarios usados durante la optimización. La búsqueda aleatoria con igual presupuesto obtuvo menor fitness medio que el AG en los tres escenarios, lo que aporta un control frente a la posibilidad de que la diferencia se explique solamente por el número de evaluaciones. La replanificación continua está implementada como extensión del prototipo, pero no se presenta aquí como evidencia cuantitativa principal. El costo computacional detallado queda pendiente de una campaña experimental específica. La literatura de scheduling vincula la reducción y

estabilización del tiempo crítico de procesamiento con una mayor robustez frente a perturbaciones [4].

## Conclusión

El presente trabajo proporciona evidencia experimental inicial sobre el uso de un Algoritmo Genético para abordar, bajo escenarios simulados, un problema de asignación de alertas en un SOC. Se modeló una representación cromosómica y una función de evaluación que integran tiempo de resolución, espera crítica, backlog, balance de carga y criticidad de activos. La evaluación sobre 20 semillas y tres niveles de demanda muestra mejoras promedio frente a Round-Robin en algunas métricas, pero los intervalos bootstrap y las pruebas pareadas no respaldan una superioridad universal. Además, la comparación de referencia con Urgencia balanceada muestra que esta regla obtiene mejor desbalance, mientras que el AG obtiene menor espera crítica. Por ello, la contribución debe interpretarse como una política de priorización y balance con un compromiso operativo explícito, no como una mejora general frente a cualquier baseline.

El modelo no mide directamente la fatiga de alertas: no representa habilidades diferenciadas, turnos, pausas ni una degradación del rendimiento por carga sostenida. En consecuencia, la fatiga se utiliza como motivación del problema y se aproxima mediante espera, backlog y cumplimiento de SLA, pero no se presenta como una variable validada experimentalmente.

Como líneas de trabajo futuro, se propone avanzar desde la automatización hacia una colaboración Humano–IA: los analistas podrían revisar las asignaciones y ajustar dinámicamente los pesos de las penalizaciones mediante retroalimentación activa. También se evaluará la robustez del cronograma frente a distribuciones de duración más diversas, cambios de disponibilidad y llegadas de mayor frecuencia. La criticidad del activo se incorpora actualmente mediante un catálogo CMDB simulado y una política global $(w_2, w_3)$; una extensión posterior deberá conectar esos pesos con un inventario real y con atributos de vulnerabilidad y dependencia. Estas líneas responden a las brechas identificadas por Jalalvand et al. [3] y a la perspectiva de robustez del SJSP estudiada por Boukedroun et al. [4].

## Agradecimientos

A la cátedra de Algoritmos Genéticos por el marco metodológico, orientación técnica y soporte brindado para el desarrollo integral de la presente investigación.

## Referencias

[1] Sharafaldin, I., Lashkari, A. H., y Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. CICIDS2017.

[2] Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.

[3] Jalalvand, F., Chhetri, M. B., Nepal, S., y Paris, C. (2024). *Alert Prioritisation in Security Operations Centres: A Systematic Survey on Criteria and Methods*. ACM Computing Surveys, 57(2), 1–36. https://doi.org/10.1145/3695462

[4] Boukedroun, M., Duvivier, D., Ait-el-Cadi, A., Poirriez, V., y Abbas, M. (2023). *A hybrid genetic algorithm for stochastic job-shop scheduling problems*. RAIRO–Operations Research, 57(4), 1617–1645. https://doi.org/10.1051/ro/2023067